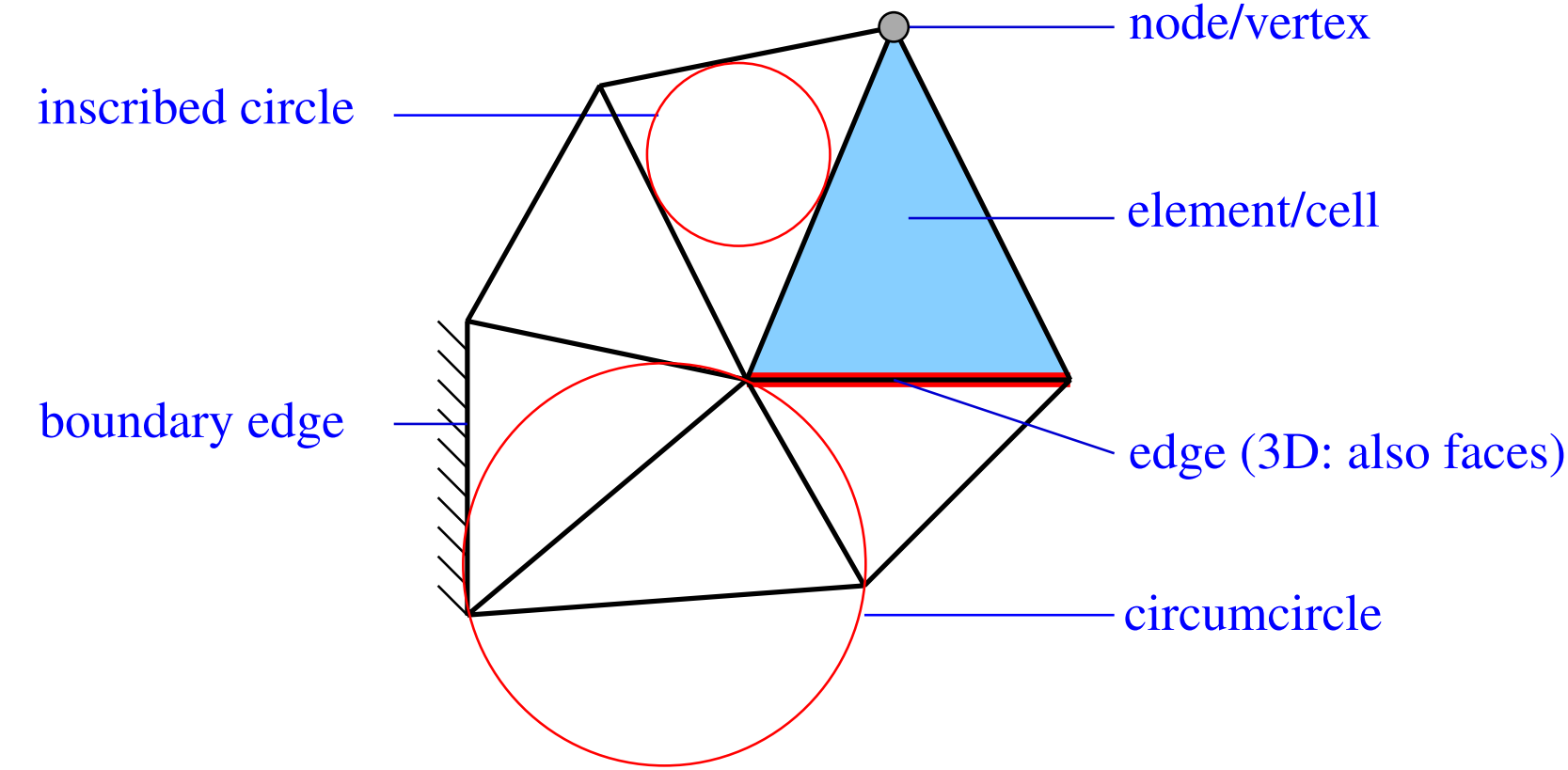
Mesh Generation

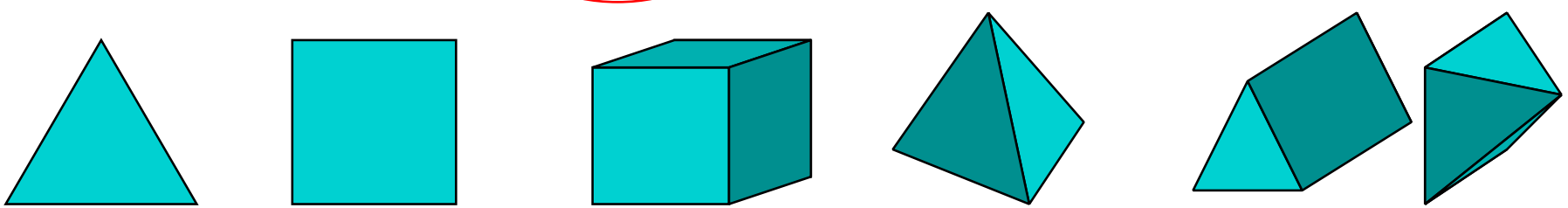
K.J. Fidkowski

Aero 623
University of Michigan

Winter 2026

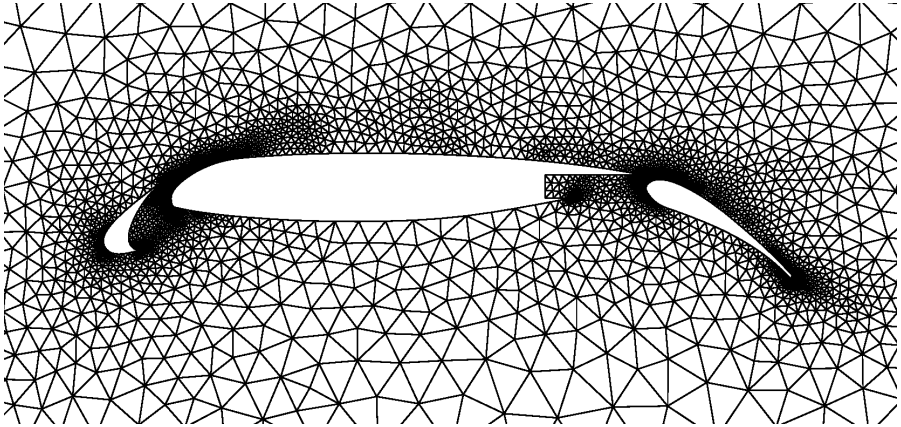
Definitions



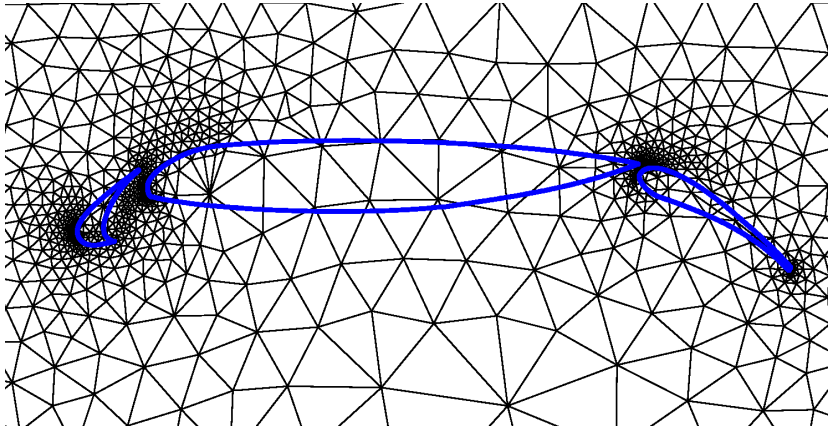


simplex elements

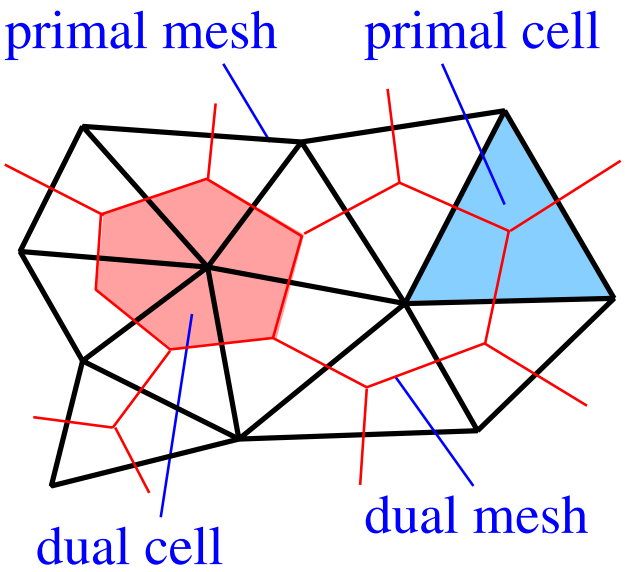
Definitions (ctd.)

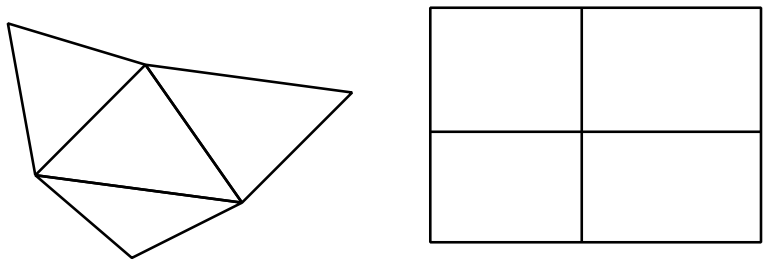


Boundary-conforming mesh

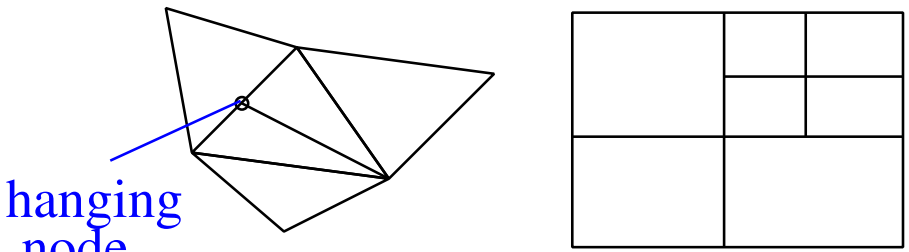


Cut-cell mesh



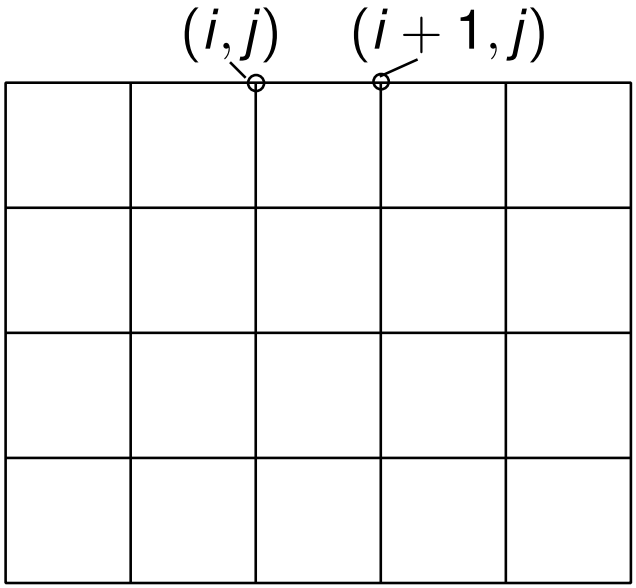


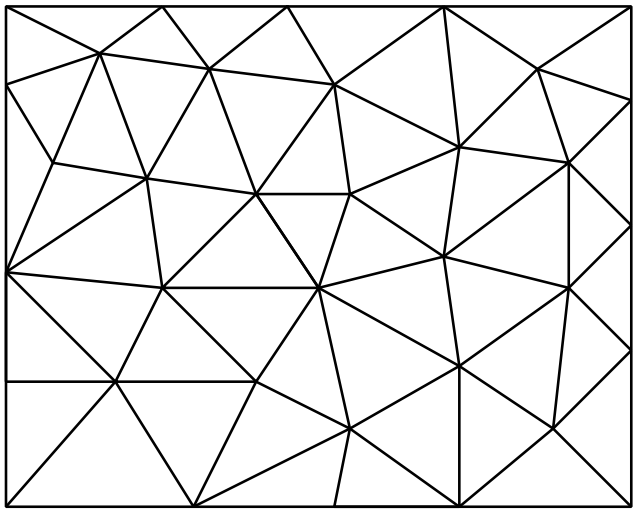
Conforming meshes



Non-conforming meshes

Two types of meshes





Structured meshes

- Implied/calculated connectivity
- Usually quads/hexes
- Usually higher accuracy
- Difficult to generate/adapt

Unstructured meshes

- Connectivity explicitly stored
- Usually triangles/tets
- Higher geometric flexibility
- More expensive to store

Unstructured meshes

- Storage
- Generation algorithms
- Size and quality
- Codes

Mesh storage

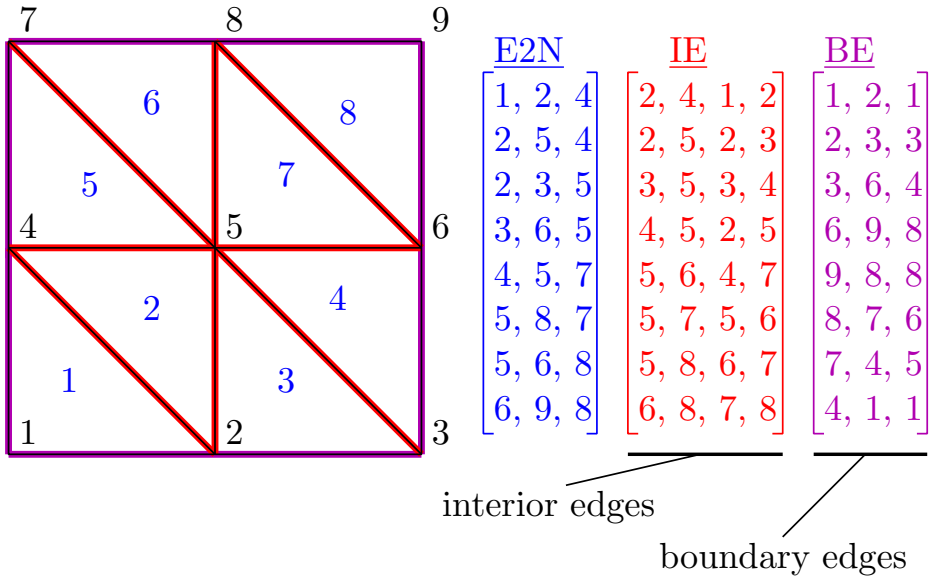
- Minimum amount of information to store an unstructured mesh [numbers shown for a 2D triangular mesh]:
  - Node coordinates [$\text{nnode} \times 2$]
  - Node numbers for each element [$\text{nelem} \times 3$]
  - Node numbers for each boundary face/edge (not needed if only one boundary exists) [$\text{nbface} \times 2$]
- More information may be needed during mesh generation
- Pre-processing is required to obtain element-to-element connectivity and boundary-edge/face-to-element connectivity for the solver
- Meshes can be big – need to avoid $\mathcal{O}(N^2)$ algorithms, where N is the number of nodes or elements

A node-hash connectivity algorithm

Given:
An "element-to-node" array, *E2N* of size $[\text{nelem} \times 3]$, for a triangular mesh.

Identify:

- # interior edges and for each: $[n_1, n_2, \text{elem}_1, \text{elem}_2]$
- # boundary edges and for each: $[n_1, n_2, \text{elem}]$



$\mathcal{O}(\text{nelem})$ algorithm:

- Loop over elements and store visited edges by node pairs (i.e. form a hash table based on the nodes)
- In this loop, an already-visited edge identifies an interior edge
- Remaining unpaired edges are boundaries

Node-hash connectivity algorithm code (Matlab)

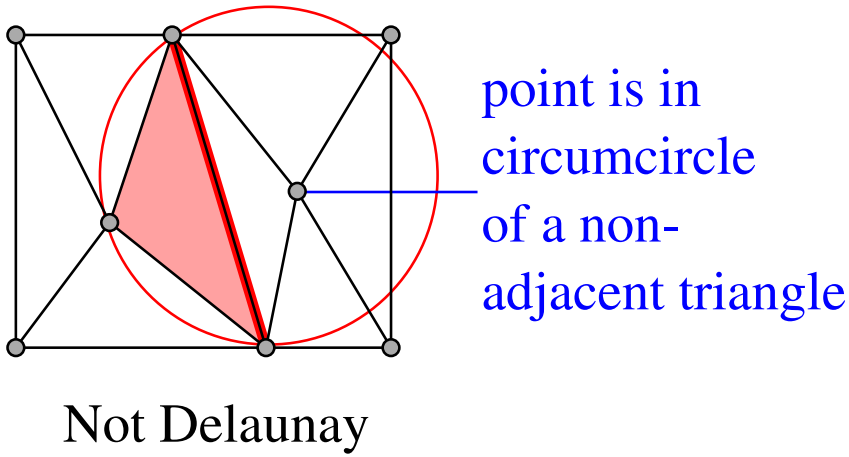
```
function [IE,BE] = edgehash(E2N)
% This function identifies interior and boundary edges, and their
% connectivities, in a triangular mesh given an element-to-node array.
%
% INPUT : E2N = [nelem x 3] array mapping triangles to nodes
% OUTPUT: IE = [niedge x 4] array giving (n1, n2, elem1, elem2)
%              information for each interior edge
%         BE = [nbedge x 3] array giving (n1, n2, elem)
%              information for each boundary edge

nelem = size(E2N,1)             % number of elements
nnode = max(max(E2N))           % number of nodes
H = sparse(nnode,nnode);        % Create a hash list to identify edges
IE = zeros(ceil(nelem*3/2), 4); % (over) allocate interior edge array
niedge = 0;                     % number of interior edges (running total)

% Loop over elements and identify all edges
for elem = 1:nelem,
  nv = E2N(elem,1:3);
  for edge = 1:3,
    n1 = nv(mod(edge  ,3)+1);
    n2 = nv(mod(edge+1,3)+1);
    if (H(n1,n2) == 0), % edge hit for the first time
      % could be a boundary or interior; assume boundary
      H(n1,n2) = elem;  H(n2,n1) = elem;
    else % this is an interior edge, hit for the second time
      oldelem = H(n1,n2);
      if (oldelem < 0), error 'Mesh input error'; end;
      niedge = niedge+1;
      IE(niedge,:) = [n1,n2, oldelem, elem];
      H(n1,n2) = -1;  H(n2,n1) = -1;
    end
  end
end

IE = IE(1:niedge,:);  % clip IE

% find boundary edges
[I,J] = find(triu(H)>0);
BE = [I, J, zeros(size(I))];
for b = 1:size(I,1), BE(b, 3) = H(I(b),J(b)); end;
```

Unstructured mesh generation algorithms

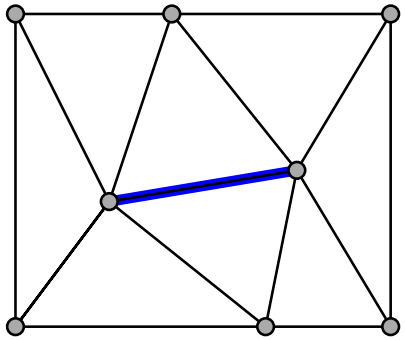
- Delaunay triangulation
  - Can be applied to an arbitrary set of points
  - Algorithms allow for efficient insertion of new points
- Advancing front
  - Propagate front of simplex elements into domain interior from a boundary mesh
  - Good for boundary layer meshes, where regularity near the boundary is required
- Grid and quadtree meshing
  - Intersect geometry with a Cartesian or quadtree-refined mesh
  - Form triangular elements by quad subdivision

Delaunay triangulation

A useful and unique triangulation of a set of arbitrary input points



Not Delaunay



Delaunay

Properties

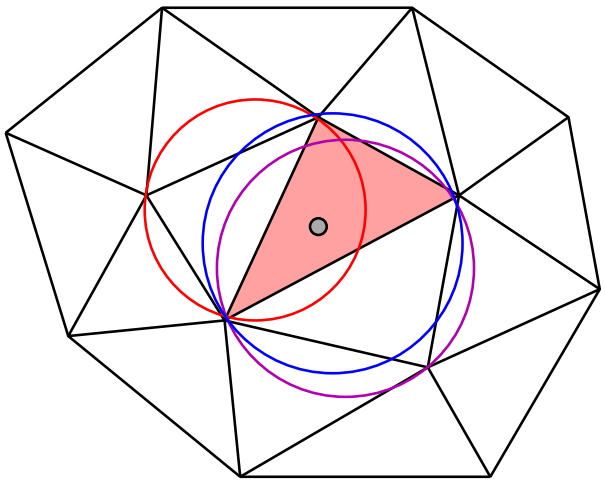
- The circumcircle of each triangle contains no other points
- Maximizes the minimum angle over all the triangles
- Dual mesh based on circumcenters is the Voronoi tessalation
- Extends naturally to 3D

Delaunay algorithms

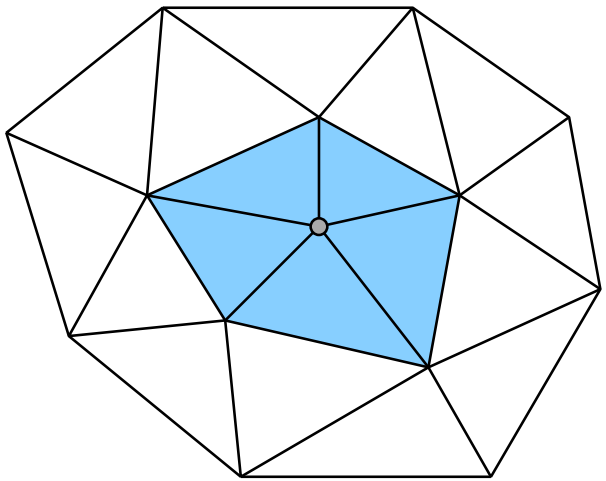
- Brute force
  - Begin with any triangulation of the given points
  - Flip edges of all triangle pairs that are not Delaunay (sum of opposite angles needs to be less than 180^o)
  - $\mathcal{O}(N^2)$ run time
- Incremental node insertion
  - Given a Delaunay triangulation of $N-1$ points, insert N^{th} point
  - Re-triangulate (small) region of affected triangles (e.g. Bowyer-Watson algorithm [2])
  - $\mathcal{O}(N \log N)$ run time
- Subdivision and merging
  - Recursively split set of points, triangulate each set, and join sets together.
  - $\mathcal{O}(N \log \log N)$ run time

Delaunay refinement

Procedure for adding resolution to a Delaunay triangulation



Before point insertion

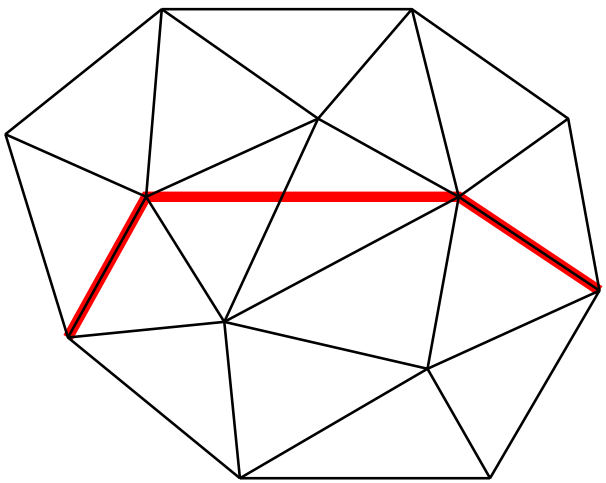


After point insertion

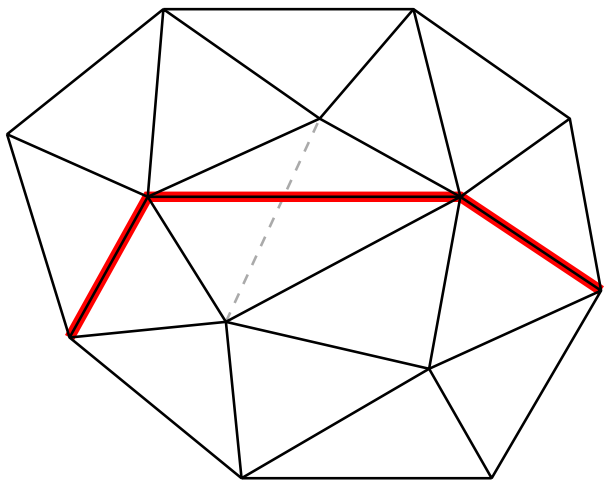
- Begin with a Delaunay triangulation of N points
- Pick a “bad” triangle: large or with undesirable angles
- Add the centroid of that triangle as the $N + 1^{\text{st}}$ point
- Re-triangulate region of affected triangles

Constrained Delaunay

- Sometimes we need a mesh with edges between certain points
- These edges could form a geometry (interior to be removed)
- A Delaunay triangulation will not necessarily respect these edges
- The desired edges can be recovered using edge swaps



Delaunay
Edge constraints not satisfied



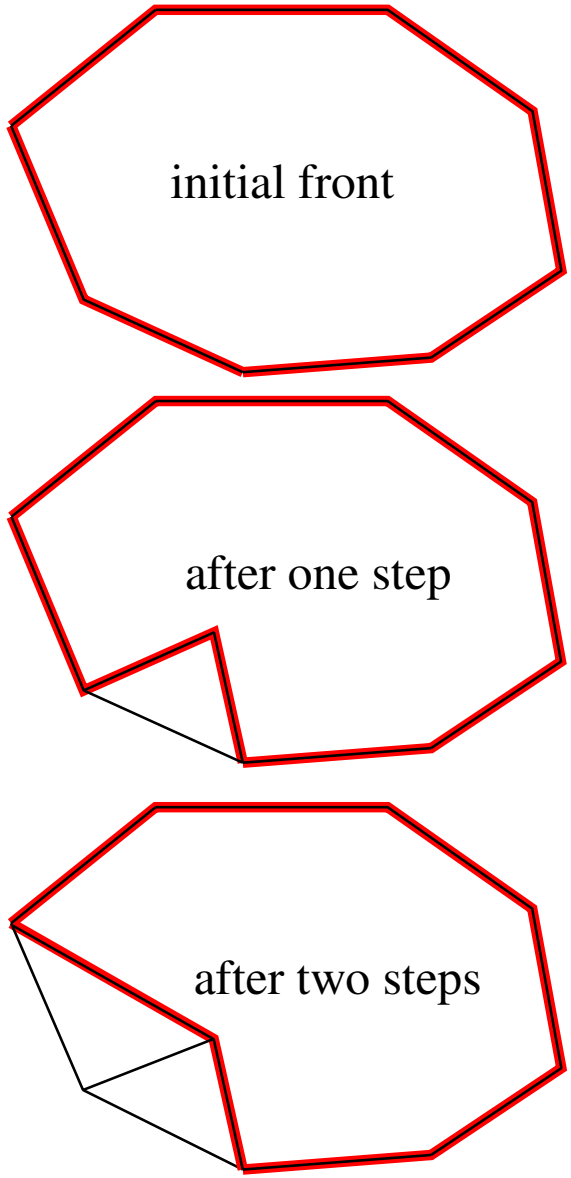
Not Delaunay
Edge constraints satisfied

The advancing front algorithm

Front: set of all edges which are currently available to form a new triangular element.

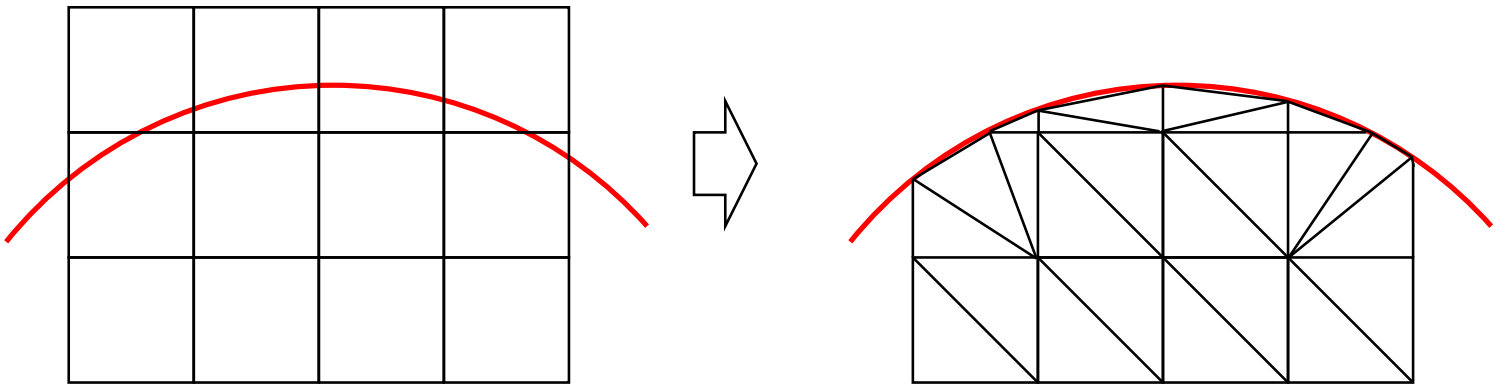
Algorithm [7, 6]

- Initialize front by distributing nodes on the boundary
- Select an edge from the current front and generate a triangle – by creating a new node or connecting existing ones – based on the desired mesh density.
- Update the front. If no more edges available (empty front) the process is finished. Else repeat the previous step.

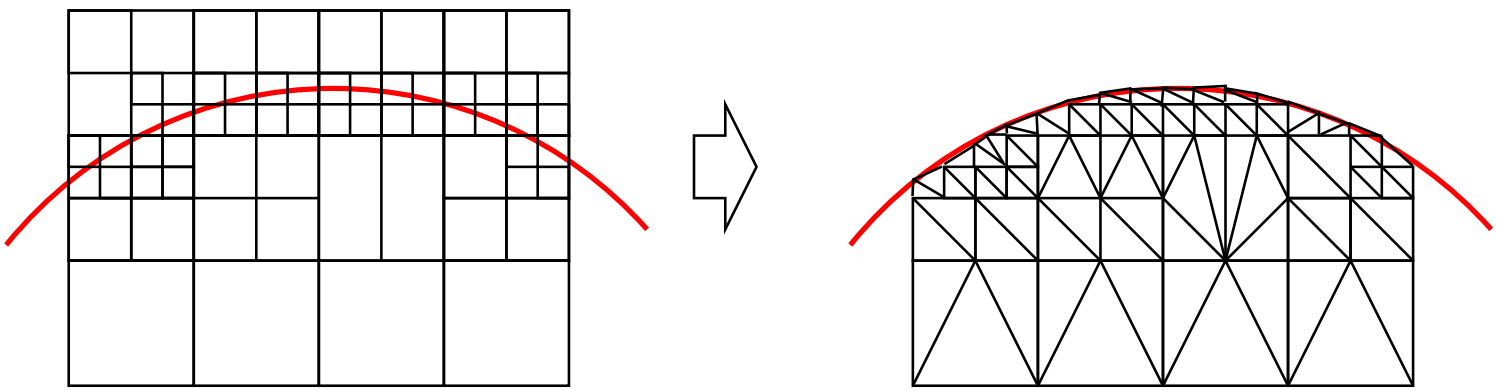


Grid and quadtree meshing

- Intersect regular Cartesian grid with geometry. Handle a few special cases at the intersections.

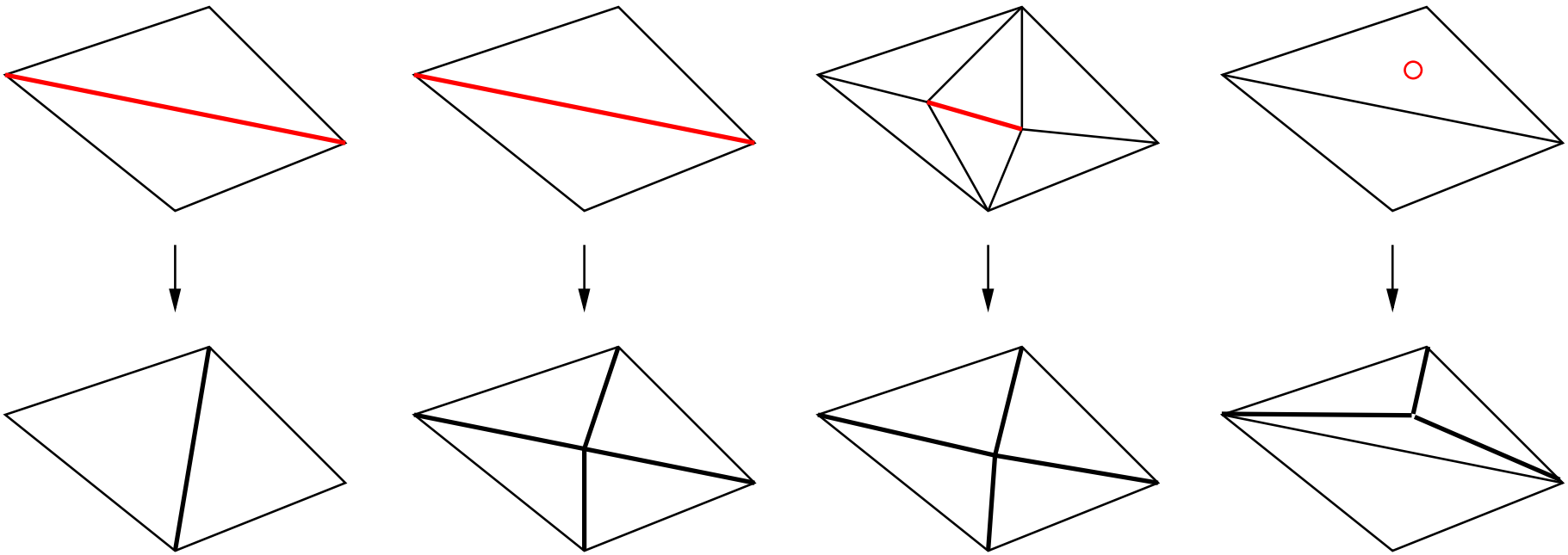


- Improve resolution by using a quadtree grid



Triangular-mesh refinement

Meshes can be refined/coarsened/modified using local operators



edge swap | edge split | edge collapse | node insertion

Mesh quality

Mesh quality guidelines

- Large angles (close to 180^o) are problematic for most discretizations
- Small angles (close to 0) are problematic for some discretizations

$$\text{One triangle quality measure: } \frac{4\sqrt{3}\,\text{Area}}{L_1^2 + L_2^2 + L_3^2} \quad (1 \text{ for equilateral})$$

- Large differences in adjacent element areas can cause conditioning and accuracy problems.

Additional considerations

- Are requested mesh size limits met?
- Is mesh anisotropy correct?
- Does a mesh look good – i.e. how smooth are area transitions?

Size control

Mesh size, h

- Typically a length scale – "h denotes element size ..."
  - incircle/circumcircle diameter
  - shortest/longest edge or altitude
  - ratio of element area to the perimeter
- Available from:
  - a priori knowledge of the solution
  - geometry features
  - adaptive indicator

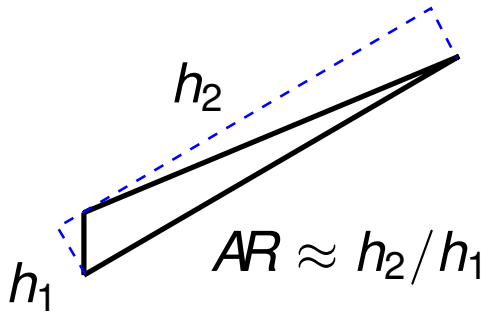
Mesh size field, $h(\mathbf{x})$

- Element size distribution
- Used to guide adaptive meshing
- Usually prescribed on a background mesh (Cartesian grid or previous adapted mesh)

Mesh anisotropy

One definition of element anisotropy, AR

Ratio of long side to short side of an oriented bounding box



Anisotropy is needed

- When the solution varies a lot in one direction and not much in the perpendicular direction: e.g. in boundary layers, wakes, and shocks
- Using isotropic elements would be very inefficient
  - $AR^{\dim-1}$ times as many boundary layer elements (sometimes 1/3 of total element count) for spatial dimension $\dim$
  - not practical for high Reynolds number flows where the average aspect ratio in the boundary layer is several hundred

Anisotropic meshing

- Produced routinely using the advancing front method for 2D and 3D boundary-layer meshes
- A newer application is fully-unstructured metric-based meshing [4]

Metric-based meshing

Idea: make an isotropic high-quality mesh, e.g. via Delaunay, under a new length measure

- The standard Euclidian measure corresponds to an identity metric in defining the distance between two points separated by $\Delta\mathbf{x}$.

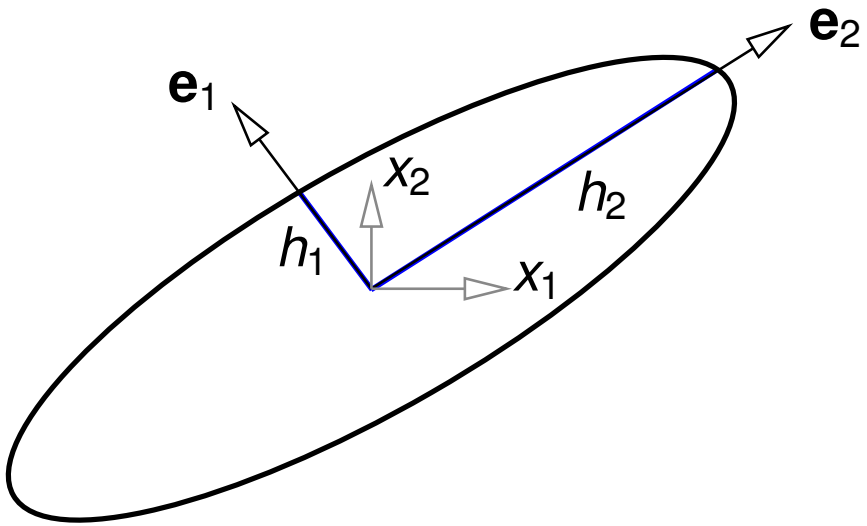
$$L_I^2 = \Delta\mathbf{x}^T\mathbf{I}\Delta\mathbf{x}$$

- An arbitrary metric is a $\dim \times \dim$ SPD matrix denoted by $\mathbf{M}$. It yields a distance measure of

$$L_M^2 = \Delta\mathbf{x}^T\mathbf{M}\Delta\mathbf{x}$$

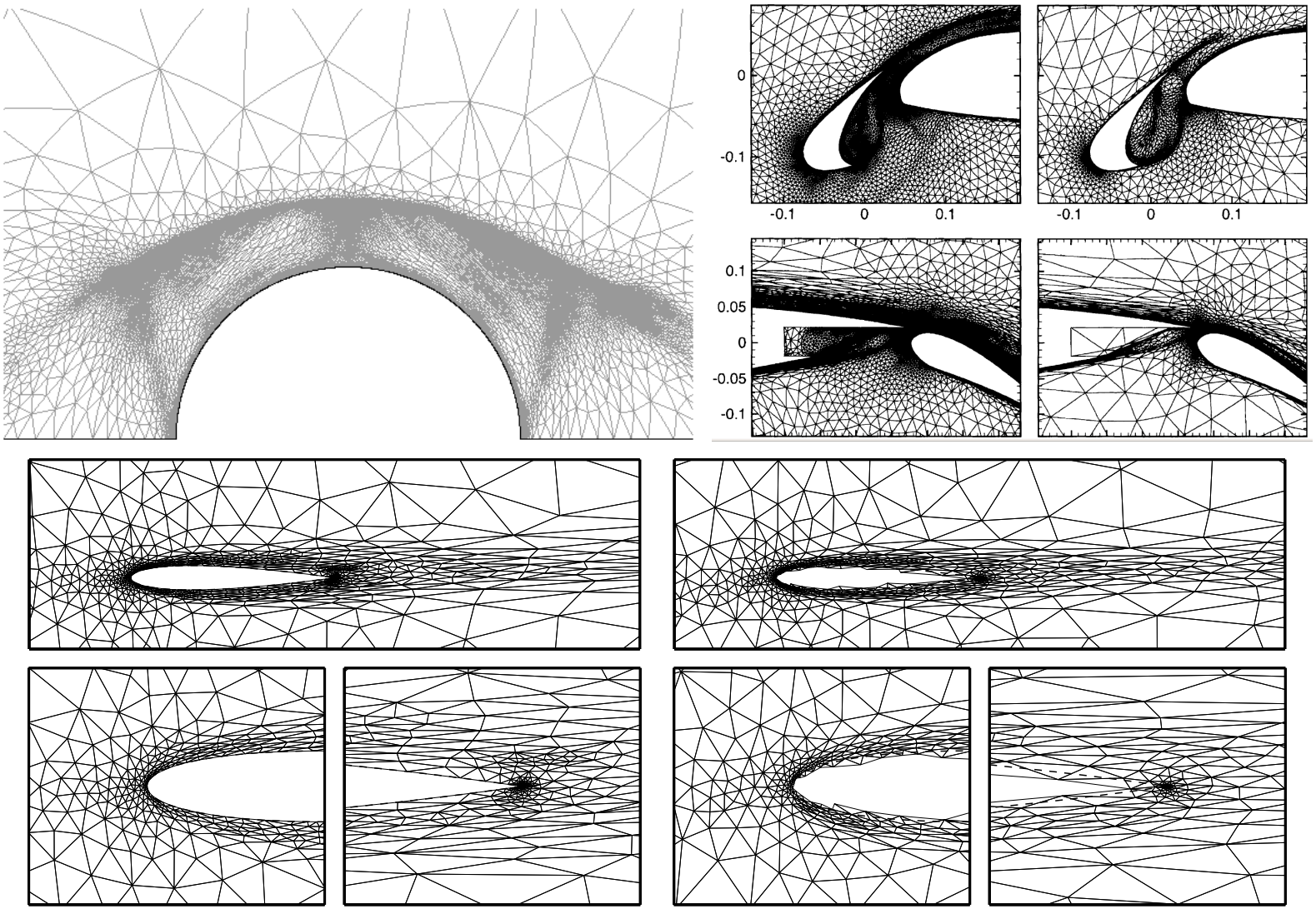
Metric properties

- unit measure requirement, $\Delta \mathbf{x}^T \mathbf{M} \Delta \mathbf{x} = 1$, describes an ellipsoid in physical space



- eigenvectors of $\mathbf{M}$, $\mathbf{e}_i$, are the principal stretching directions
- eigenvalues of $\mathbf{M}$, λ_i, are related to the principal stretching lengths (h_i) via $h_i = 1/\sqrt{\lambda_i}$
- useful for adaptive meshing where metric is obtained from some kind of error estimate (output, interpolation, etc)

Metric-based meshing examples



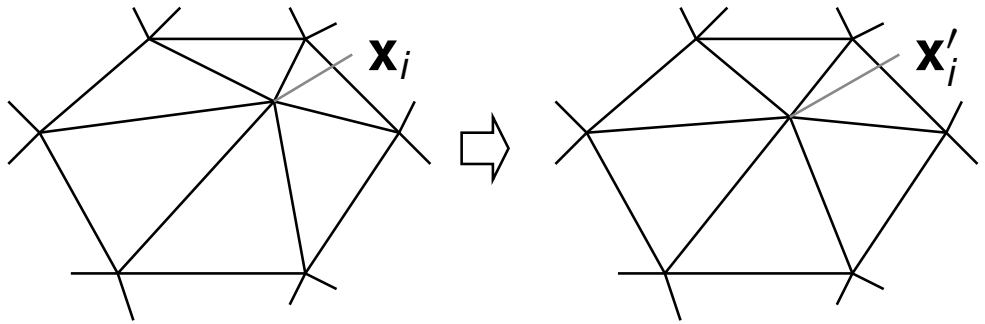
Generated using BAMG

Mesh smoothing

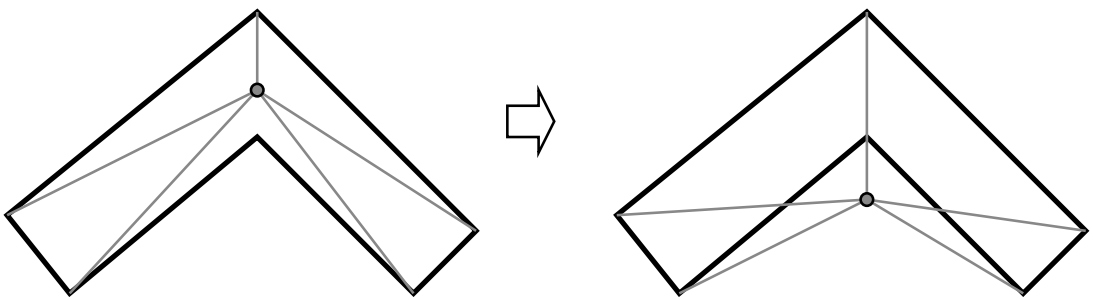
Mesh quality can be improved by "smoothing" – iteratively repositioning the internal nodes (a form of *r*-refinement) according to

$$\mathbf{x}_i' = \frac{1}{1+N_i}\left[\mathbf{x}_i + \sum_{j=1}^{N_i} \mathbf{x}_j\right]$$

N_i = # nodes adjacent to $\mathbf{x}_i$



- Often used after quadtree or Delaunay meshing
- Does not guarantee quality improvement or even mesh validity



Unstructured mesh generation codes

- Gambit (commercial, ANSYS)
- BAMG [1] = Bi-dimensional Anisotropic Mesh Generator (INRIA); C++; now part of freeFEM; `www.freefem.org`
- DistMesh (Per-Olof Persson, Berkeley); Matlab, distance-function-based; `persson.berkeley.edu/distmesh/`
- Gmsh (Geuzaine + Remacle); `geuz.org/gmsh/`
- Extensive list at: `www.robertschneiders.de/meshgeneration/software.html`

Structured meshes

- Motivation
- Block topologies
- Single-block mappings

Motivation

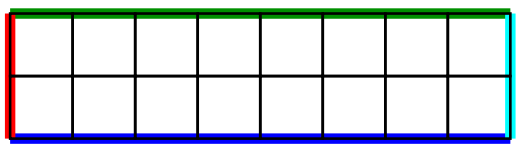
Structured grids yield:

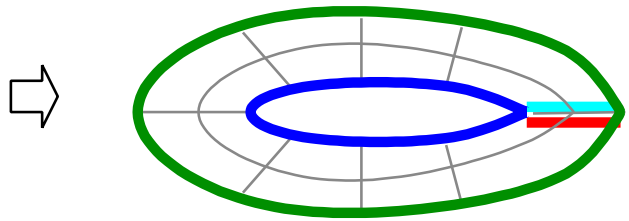
- Accurate and numerical methods
- More efficient memory and CPU usage
- Straightforward refinement for convergence studies
- Excellent quality anisotropic elements
- Geometric flexibility via multi-block approach

Single-block topologies

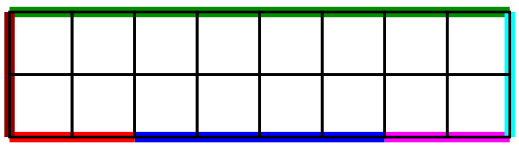
- Constructed via a mapping between a rectangular reference domain and the physical domain.
- Limited to simple geometries
- Some flexibility in mesh size control

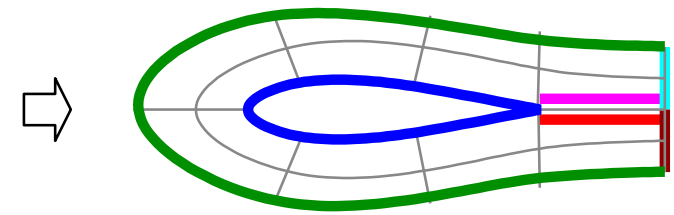
O grid:



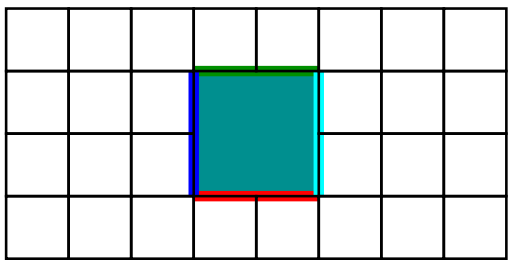


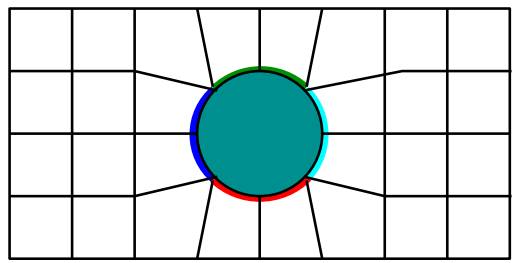
C grid:





H grid:

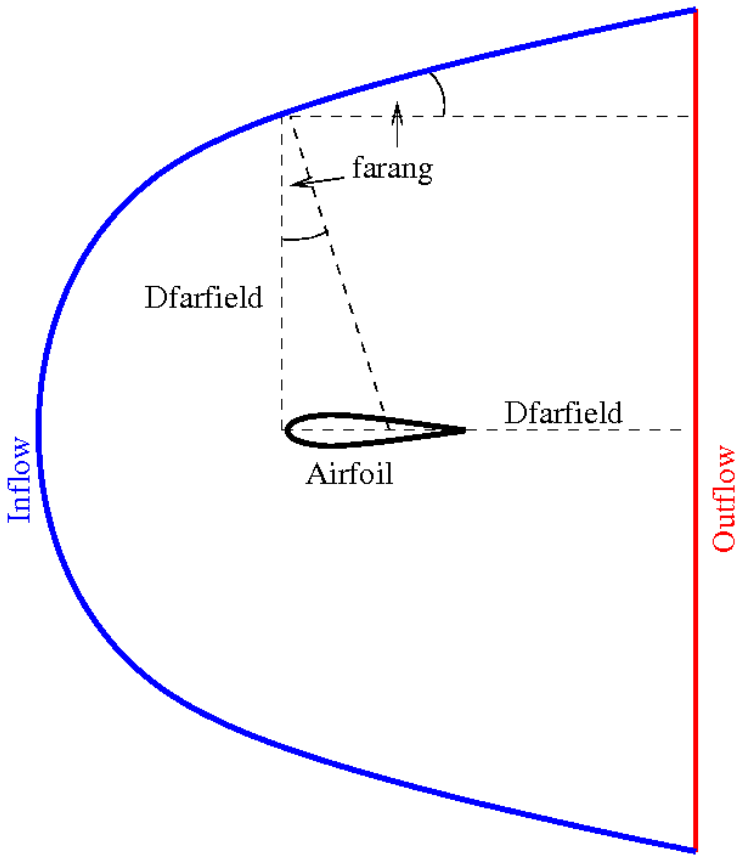


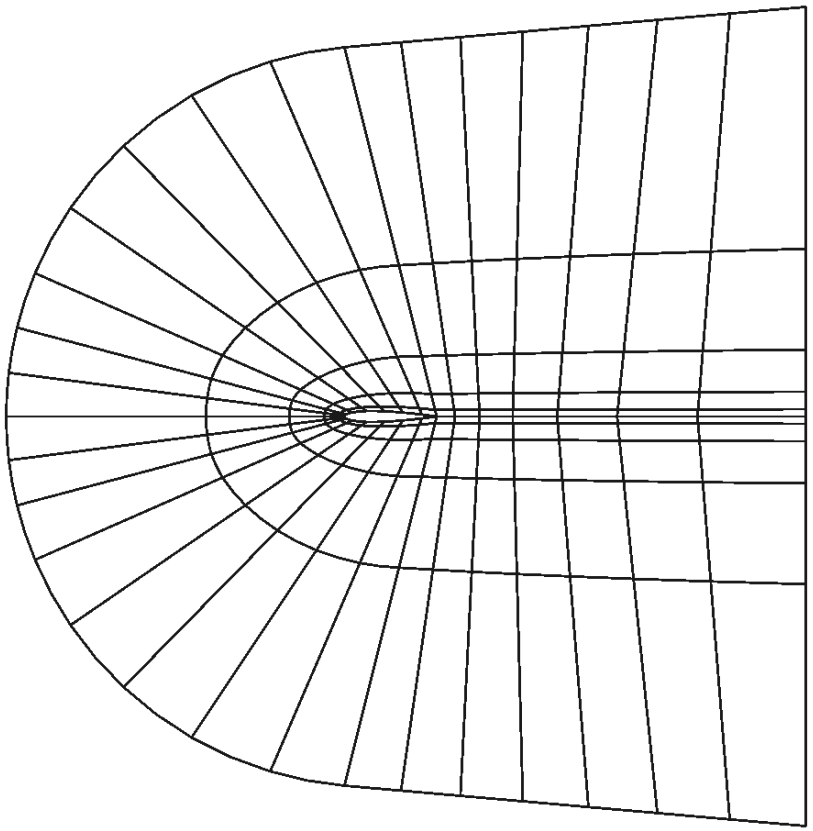


C-grid example

`make_airfoil.m`:

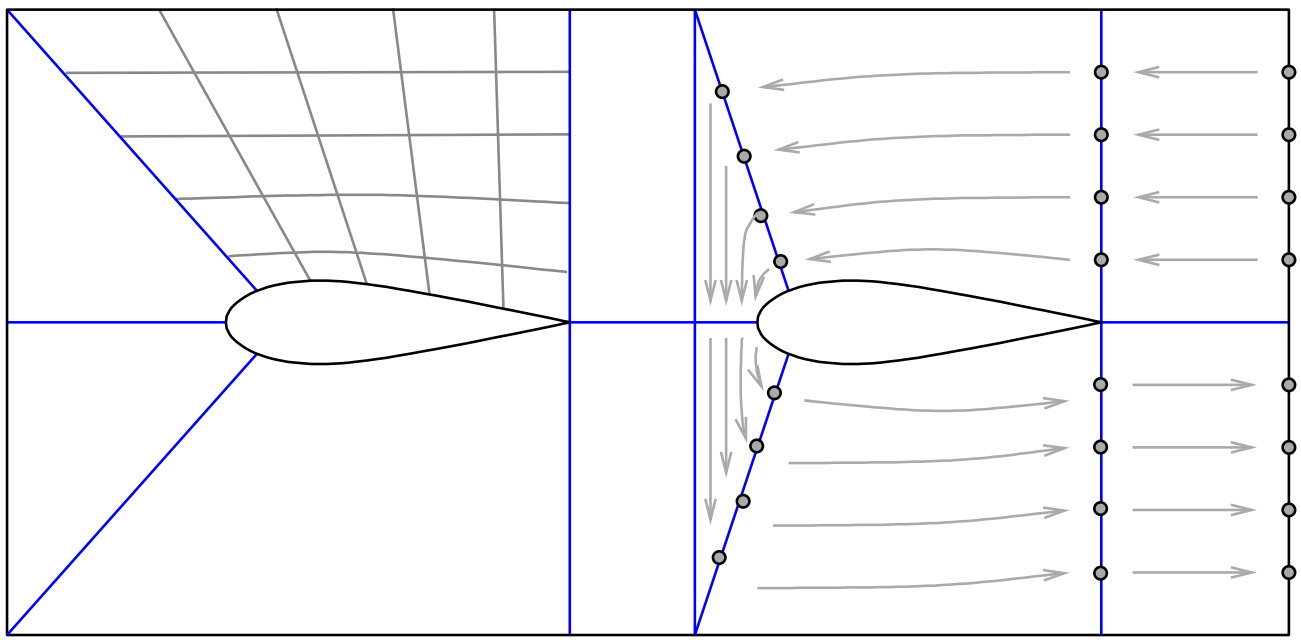
an airfoil mesh generator with mesh size/anisotropy control



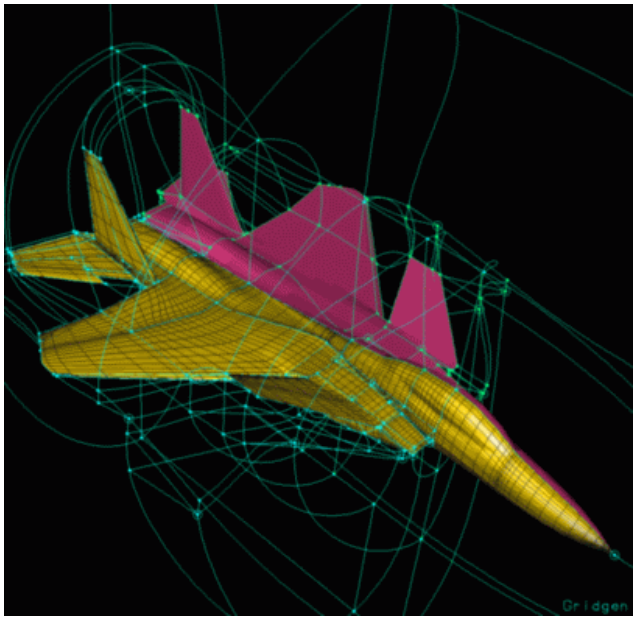


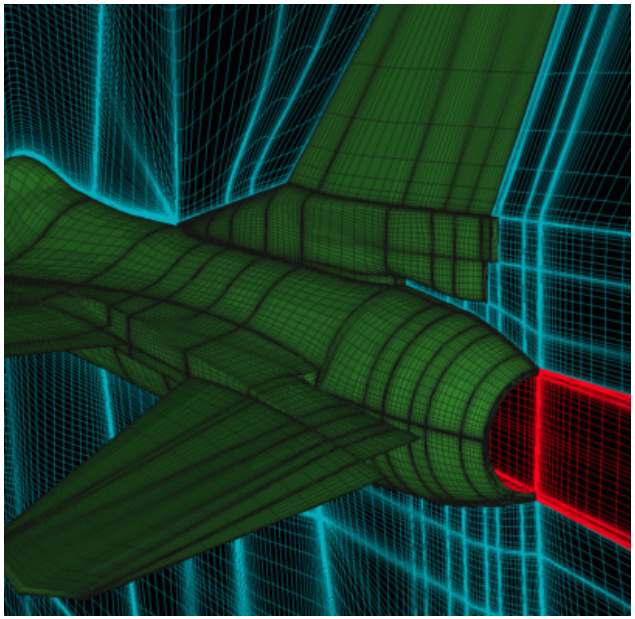
Multiblock grid generation

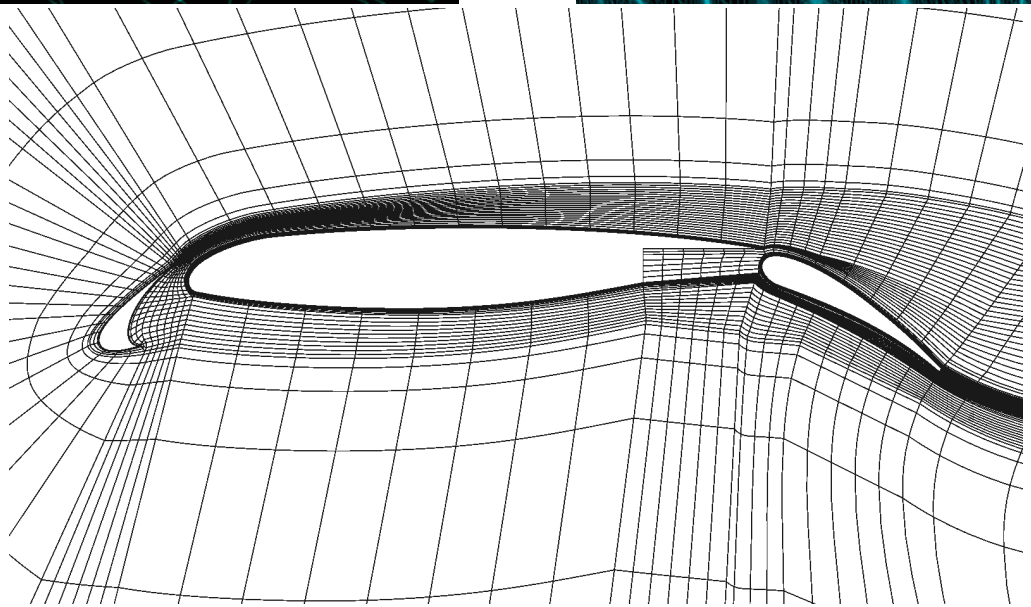
- Divide domain into a coarse “blocking” of quadrilaterals/hexahedra
- Mesh edges/faces
  - Number of points must be consistent across blocks
  - Flexibility in point coordinates
- Use single-block mappings to mesh block interiors given boundary meshes



Multiblock mesh examples

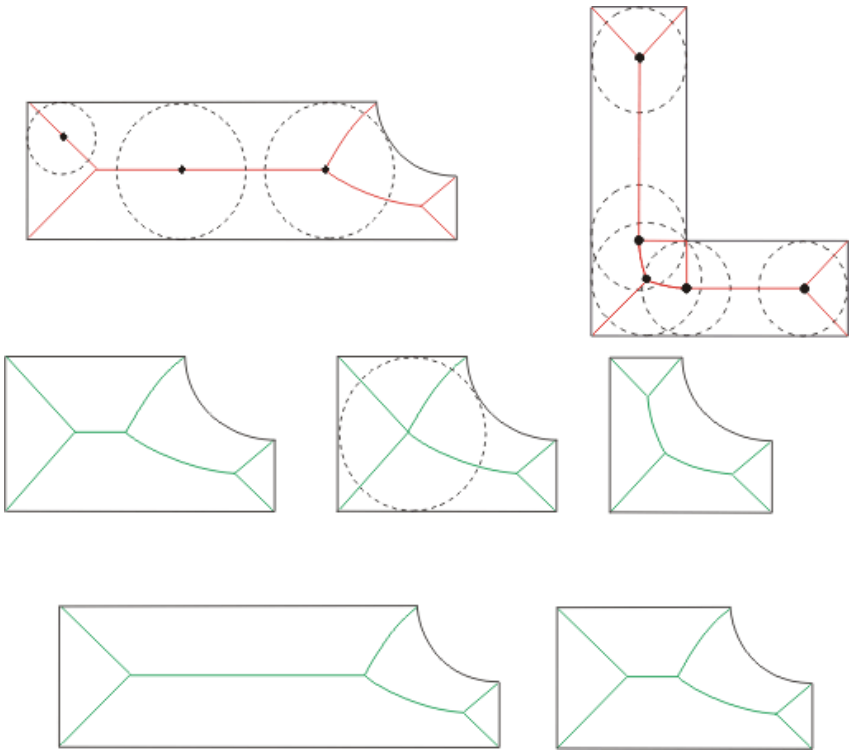






Multiblock topologies

- Generating quality blockings is difficult
- Large portion of mesh generation time devoted to this
- Blocking still requires heavy user involvement
- One automation possibility: medial-axis transform [5]



Single-block mappings

- Conformal
  - High-quality orthogonal meshes
  - Limited to two dimensions
- Algebraic
  - General and simple to implement
  - Grid quality not guaranteed
- PDE-based
  - Most-widely used approach
  - Typically generates high-quality grids

Conformal mapping

- Defined by an analytic function $z = f(\zeta)$ with $f'(\zeta) \neq 0$
  - $z = x + iy, \zeta = \xi + i\eta$ are complex numbers
  - *analytic* means a unique f' exists
- $x(\xi, \eta)$ and $y(\xi, \eta)$ satisfy the Cauchy-Riemann equations and hence are harmonic:

$$\frac{\partial^2 x}{\partial \xi^2} + \frac{\partial^2 x}{\partial \eta^2} = 0, \qquad \frac{\partial^2 y}{\partial \xi^2} + \frac{\partial^2 y}{\partial \eta^2} = 0$$

- Preserves angles
- Yields high-quality grids
- Only applicable in two dimensions

Conformal mapping examples

- Joukowski: Maps a circle to a cusped airfoil

$$z = \zeta + \frac{1}{\zeta}$$

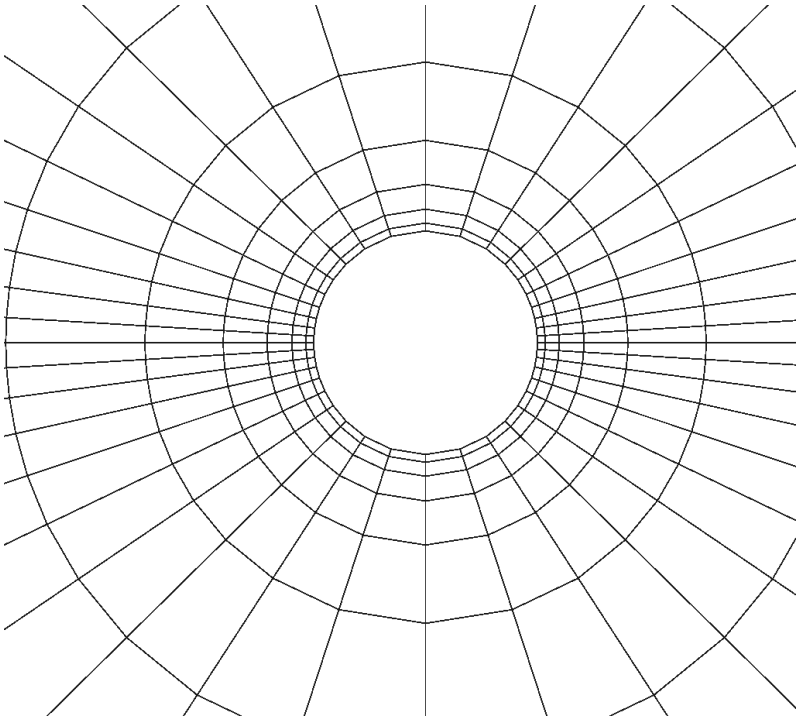
- Karman-Trefftz: Maps a circle to an airfoil with a nonzero trailing edge angle

$$\frac{z-n}{z+n} = \left(\frac{\zeta - 1}{\zeta + 1}\right)^n$$

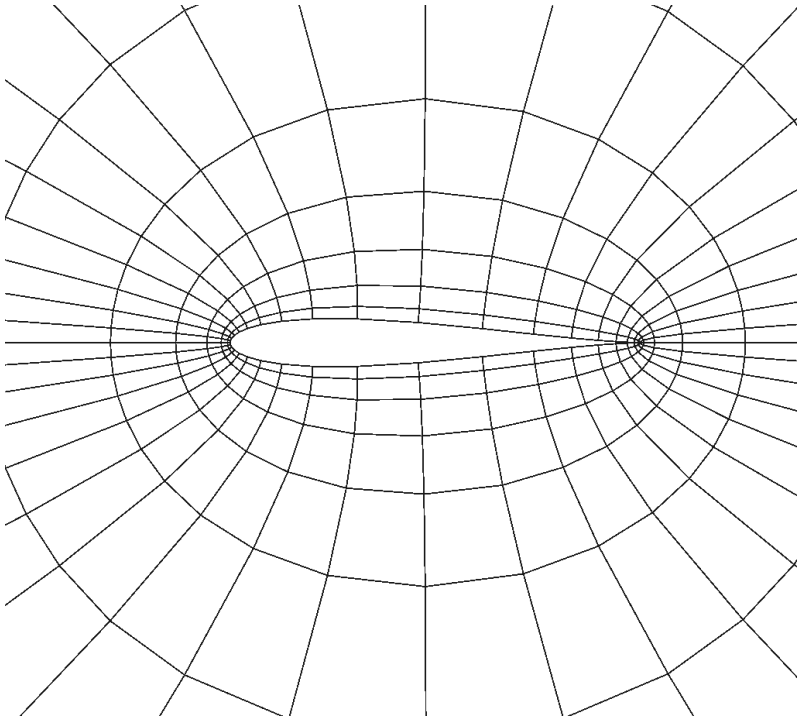
- Schwarz-Christoffel mappings:
  - Map a regular polygon into a half plane
  - Have been extended to multiply-connected domains [3]

A Joukowski-transform mesh generator

`joukowski_airfoil.m`:
Applies Joukowski transformation to a mesh around a circle



Grated mesh around a circle



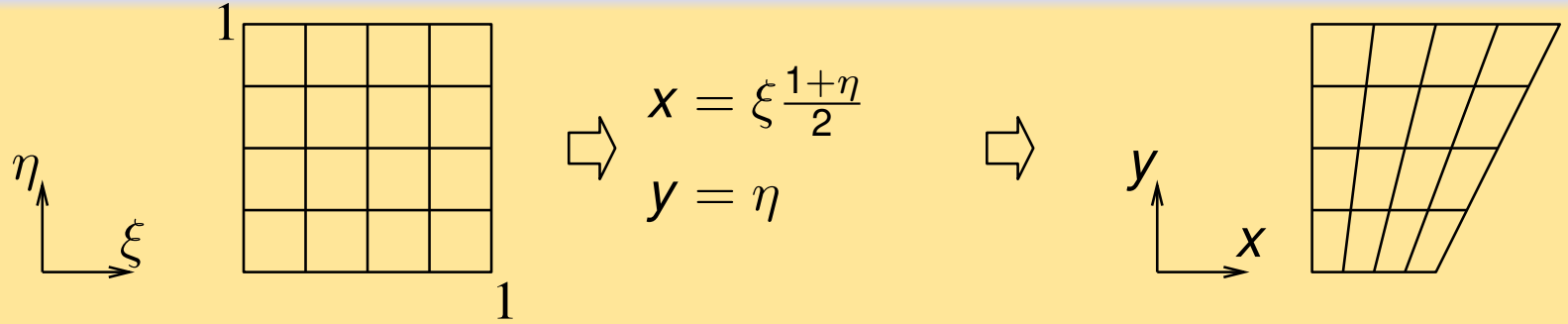
Mesh around a Joukowski airfoil

Note that 90^o *angles are preserved*

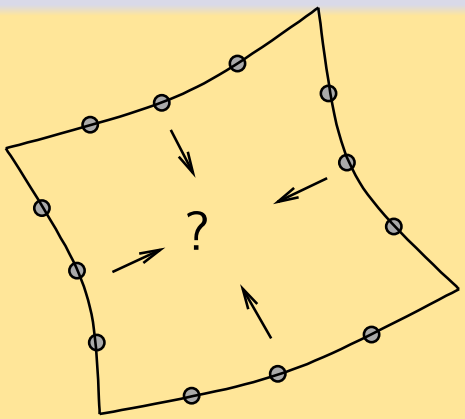
Algebraic mapping

ξ, η reference-space coordinates
x, y physical-space coordinates

Simple form: Functional mapping: $x = x(\xi, \eta)$, $y = y(\xi, \eta)$

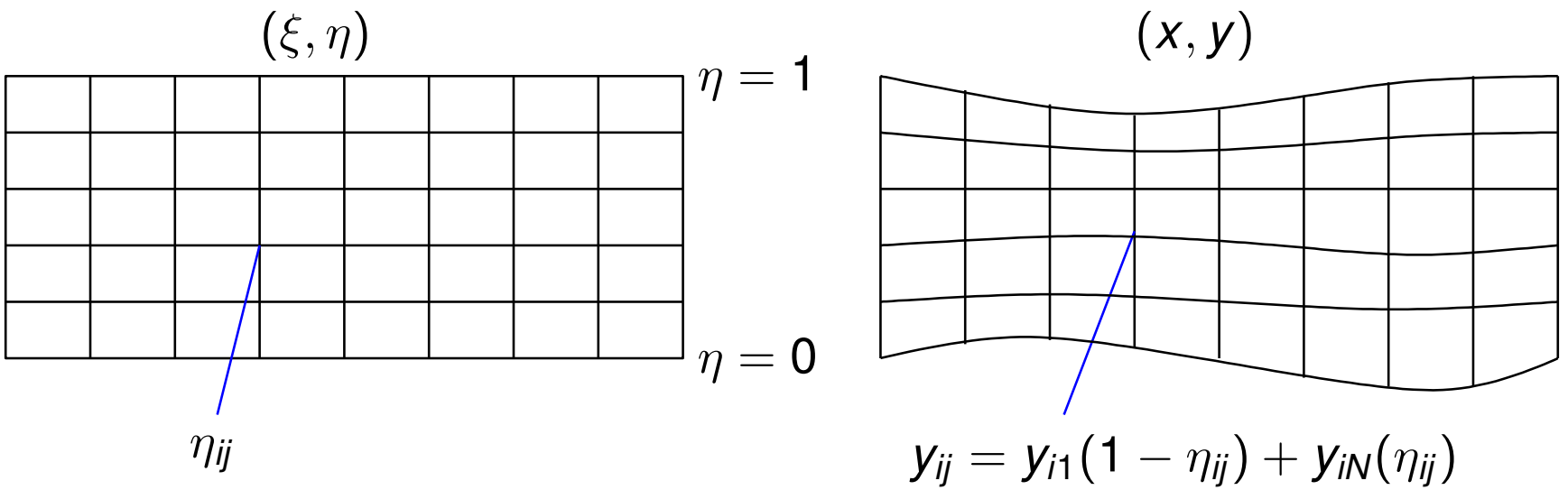


General form: Interpolation of boundary maps



Example: variable-height channel

- Choose a reference domain such that $x = \xi$
- Given coordinates of lower and upper edge points, y_{i1} and y_{iN}
- Interpolate these to obtain interior coordinates

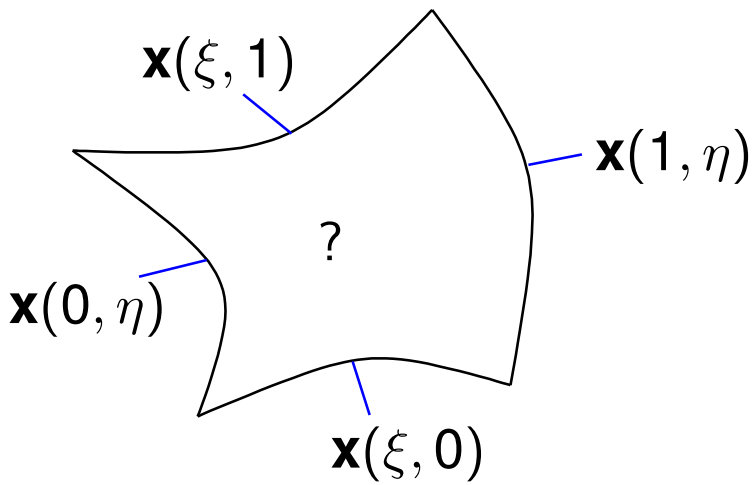


Transfinite interpolation

- Technique for mapping boundary points to interior points
- Domain must be topologically equivalent to a square (cube in 3D)
- Reference domain is taken as the unit square/cube
- Fast and general
- Not guaranteed to be one-to-one or orthogonal

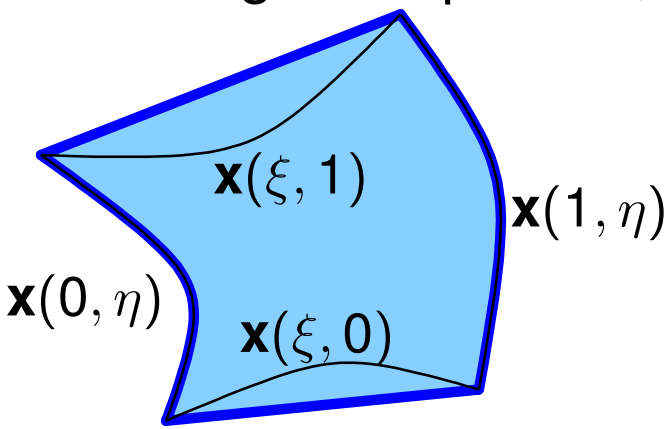
Goal: Determine $\mathbf{x}(\xi, \eta)$ given (in 2D)

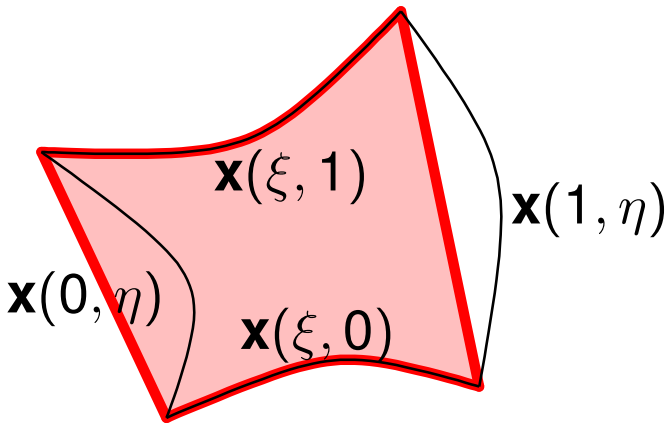
$\xi \in [0, 1]$
$\eta \in [0, 1]$



Transfinite interpolation (ctd.)

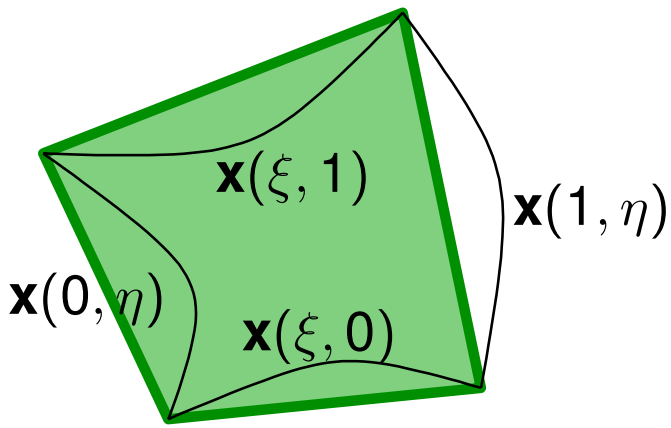
Define 1D edge interpolants,





$$\Pi_\xi \mathbf{x} \equiv (1-\xi)\mathbf{x}(0,\eta) + \xi\mathbf{x}(1,\eta) \qquad \Pi_\eta \mathbf{x} \equiv (1-\eta)\mathbf{x}(\xi,0) + \eta\mathbf{x}(\xi,1)$$

and a bilinear interpolation operator of the four nodes,

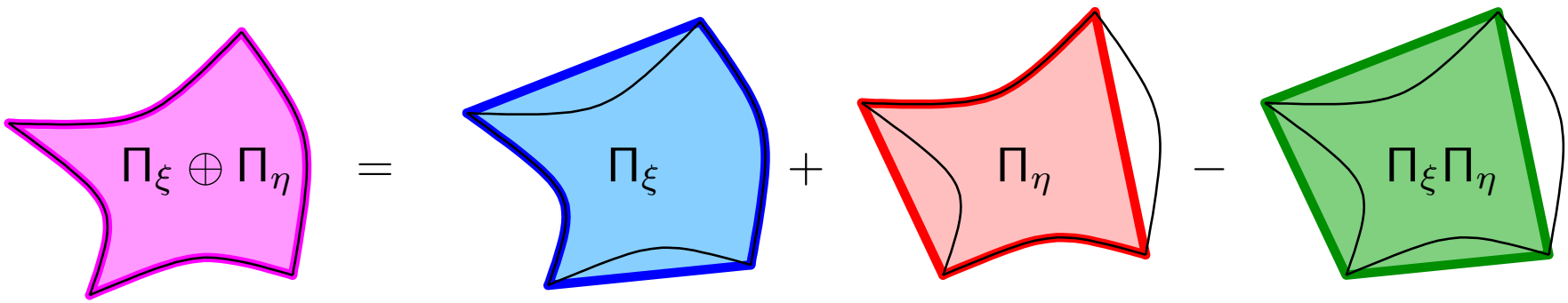


$$\Pi_\eta \Pi_\xi \mathbf{x} \equiv (1-\xi)(1-\eta)\mathbf{x}(0,0) + (1-\xi)\eta\mathbf{x}(0,1) + \xi(1-\eta)\mathbf{x}(1,0) + \xi\eta\mathbf{x}(1,1)$$

Transfinite interpolation (ctd.)

The transfinite interpolation operator is given by the boolean sum of the two edge interpolation operators,

$$\boxed{\text{Interior } \mathbf{x} = (\Pi_\xi \oplus \Pi_\eta)\,\mathbf{x} = (\Pi_\xi + \Pi_\eta - \Pi_\eta\Pi_\xi)\,\mathbf{x}}$$



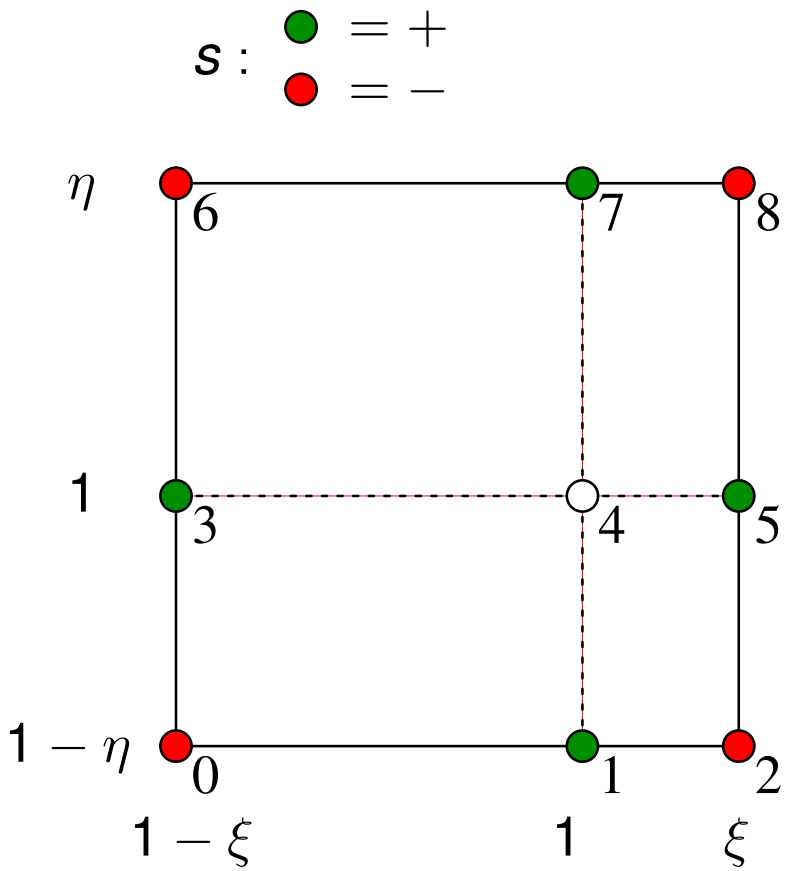
That is, the interior map is a blended combination of the edge maps that does not disturb the edge maps.

Transfinite interpolation code

C code snippet

- $X = \xi, Y = \eta$
- $x[9][2]$ contains physical coordinates
- $R[k]$ = interpolation weights on other nodes

```
// construct interpolation coefficients
XV[0] = 1.-X; XV[1] = 1.; XV[2] = X;
YV[0] = 1.-Y; YV[1] = 1.; YV[2] = Y;
for (j=0, k=0; j<3; j++)
  for (i=0; i<3; i++)
    R[k++] = YV[j]*XV[i];
R[4] = 0.;

// interpolate center node
for (d=0; d<dim; d++)
  for (k=0, x[4][d]=0.0, s=-1; k<9; k++, s=-s)
    x[4][d] += R[k]*x[k][d]*s;
```



PDE-based mapping

- Seek a mapping for which

$$\begin{aligned} \nabla^2 \xi(x,y) &= P(x,y) \\ \nabla^2 \eta(x,y) &= Q(x,y) \end{aligned}$$

  P and Q are used for grid size control. Note, the Laplace operator is with respect to x, y.

- Assuming an invertible mapping allows us to transform these equations to ones for x and y in terms of ξ and η,

$$\begin{aligned} a x_{\xi\xi} - 2b x_{\xi\eta} + c x_{\eta\eta} &= 0 \\ a y_{\xi\xi} - 2b y_{\xi\eta} + c y_{\eta\eta} &= 0 \end{aligned}$$

$$a = x_\eta^2 + y_\eta^2, \quad b = x_\xi x_\eta + y_\xi y_\eta, \quad c = x_\xi^2 + y_\xi^2$$

- These nonlinear Thompsons's equations are typically solved using an iterative method (e.g. SOR)
- For example, a finite-difference discretization on a regular grid in (ξ, η) yields the desired (x, y) coordinates of a transformed mesh

References

[1] H. Borouchaki, P. George, F. Hecht, P. Laug, and E Saltel.
Mailleur bidimensionnel de Delaunay gouverné par une carte de métriques. Partie I: Algorithmes.
INRIA-Rocquencourt, France. Tech Report No. 2741, 1995.

[2] A. Bowyer.
Computing Dirichlet tessellations.
The Computer Journal, 24(2):162–166, 1981.

[3] James Case.
Breakthrough in conformal mapping.
SIAM News, 41(1), 2008.

[4] M. J. Castro-Diaz, F. Hecht, B. Mohammadi, and O. Pironneau.
Anisotropic unstructured mesh adaptation for flow simulations.
International Journal for Numerical Methods in Fluids, 25(4):475–491, 1997.

[5] Pascal Jean Frey and Paul-Louis George.
Mesh Generation: Application to Finite Elements, Second Edition.
Wiley, New Jersey, 2010.

[6] Rainald Löhner.
Extensions and improvements of the advancing-front grid generation technique.
Communications in Numerical Methods in Engineering, 12(10):683–702, 1996.

[7] J. Peraire, M. Vahdati, K. Morgan, and O. C. Zienkiewicz.
Adaptive remeshing for compressible flow computations.
Journal of Computational Physics, 72(2):449–466, 1987.